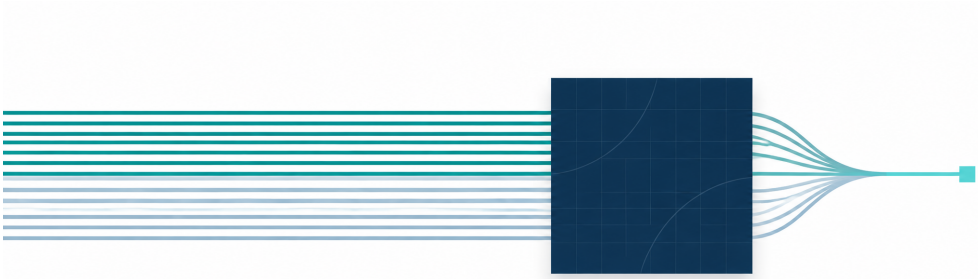
香山昆明湖 V2

自定义向量点积指令

第一阶段执行日志与过程文档

技术报告



AI 生成概念插图；不表示实际芯片版图或实测数据。

工程组合： 0f ＋ firstdot V2 ＋原 C8

团队名称： 待团队确认

成员： 待团队确认

学校： 待团队确认

材料日期： 2026 年 11 月 2 日

图像修订： 2026 年 11 月 5 日

已补齐原 9 处空图位；身份信息与波形补证范围见正文。

目录

第一阶段执行日志与过程文档 1

1.1 实验目的 1

1.2 环境与部署过程 2

1.3 hello 与 vadd 验证结果 3

1.4 波形与执行过程 4

1.5 结论及附件 6

1.6 实际执行日志摘录 6

1.6.1 构建和运行时间记录 7

1.6.2 程序构建 7

1.6.3 差分运行与结果 7

1.7 从原波形提取的事件时间线 8

1.8 日志附件索引 8

1.9 工程实现与验证方法补充 9

1.9.1 部署任务的依赖关系 9

1.9.2 内存、swap 与并发的工程含义 9

1.9.3 固定源码与依赖恢复 10

1.9.4 程序装载、启动与串口输出 10

1.9.5 启动差分故障的定位过程 10

1.9.6 向量加法结果的解释 11

1.9.7 日志附件的阅读和复查顺序 12

1.9.8 环境复现的输入层级与检查顺序 12

1.9.9 启动一致性问题的反事实分析 13

1.9.10 向量用例如何支持结论 13

1.9.11 构建资源与失败证据的管理 14

1.9.12 阶段一成果向后续阶段的传递 14

1.9.13 故障定位时的观察边界 15

1.9.14 可重现结果与不可推广结果 15
1.9.15 对阶段二验证设计的具体启示 16
1.9.16 阶段一验收记录的阅读方式 16

1.10 环境部署详细方法 17
1.10.1 Windows/WSL 原生文件系统与工具分工（工程实践） 17
1.10.2 WSL 内存 18
1.10.3 固定源码 19
1.10.4 已验证工具 19
1.10.5 新机器安装 20
1.10.6 使用现有环境 21

1.11 NEMU 构建原始日志 21

1.12 EMU 构建原始日志 24

1.13 工具与编译链版本原始输出 25

1.14 固定子模块版本清单 26

1.15 原始波形事件记录 29

1.16 向量运算单元事件记录 30

第一阶段执行日志与过程文档

工程组合：0f ＋ firstdot V2 ＋原 C8　材料日期：2026 年 11 月 2 日

赛题为面向边缘 AI 大语言模型推理的 RISC-V 自定义 vdot 指令设计。本文初稿日期为 2026 年 11 月 2 日，阶段一原实验日期为 2026 年 10 月 9 日，最终工程版本按《最终版本确认》绑定。

1.1　实验目的

本阶段建立香山硬件仿真、NEMU 参考模型与裸机程序之间的协同验证环境。以 hello 输出检查程序启动和串口链路，以向量加法检查向量状态、访存、执行、写回及提交过程，为后续自定义点积指令提供基线。

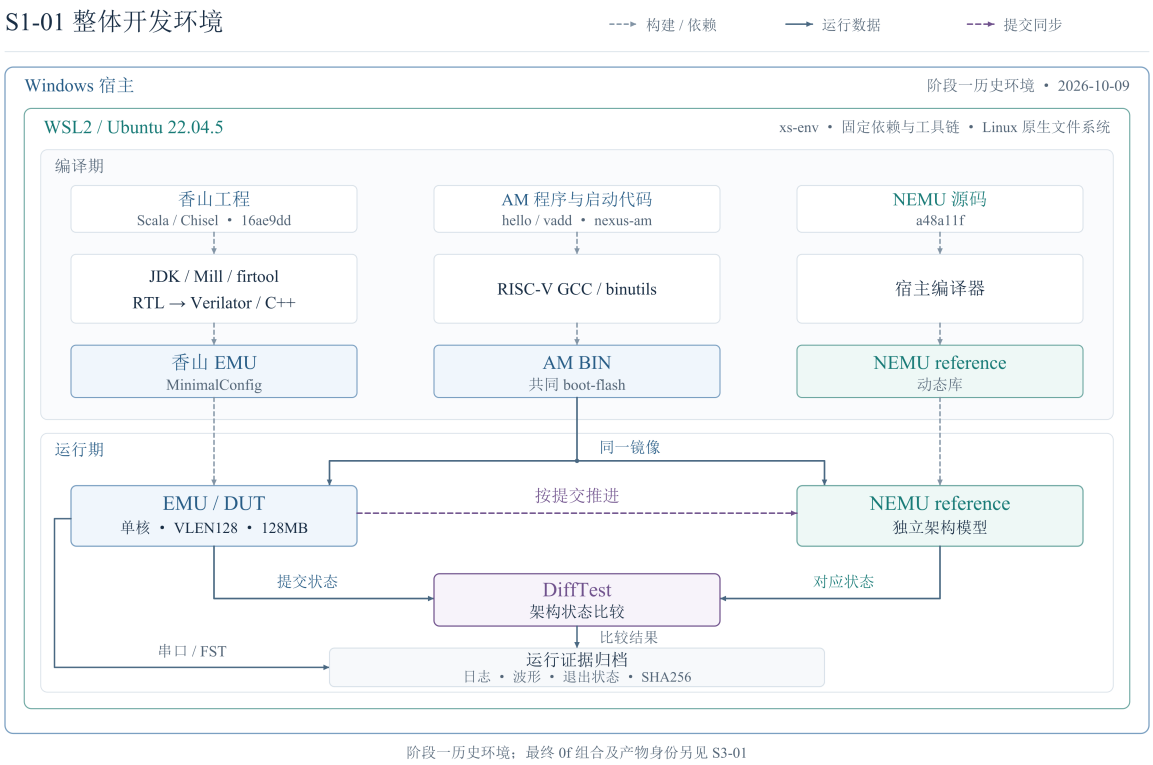


图 1-1　开发环境与软硬件协同验证框架

注：示意图展示 Windows/WSL 环境中 RTL、AM 程序、NEMU 与 DiffTest 之间的构建依赖和运行数据流。阶段一历史环境日期为 2026-10-09。

来源与范围：用户提供示意图；依据：环境记录、team/scripts/stage1.sh 及已固定版本表。版本范围：阶段一历史环境；与最终 0f 组合分别标注。

1.2 环境与部署过程

实验使用 Windows 下的 WSL2 Ubuntu 22.04.5，处理器为 Ryzen 7 7840H。原机器物理内存约 27.7GiB，WSL 配置 20GB 内存及 16GB swap。硬件与仿真工作放在 WSL 原生文件系统。

表 1-1 环境与部署过程

组件	固定版本或设置
xs-env	b0b506481e839a5df8a580e76f5aa246dc60fe7c
原香山基线	16ae9ddcda54fc9a2fddffad73174cf793ac7814
原 NEMU 基线	a48a11f208ff81ad765591d83271d1b339fb3861
nexus-am	086a238062fdfb3281b745a537b361983d0154b5
JDK / Mill / firtool	21.0.12 / 0.12.3 / 1.62.1
Verilator	5.052，固定 ea338be 提交
RISC-V GCC / binutils	11.4.0 / 2.38
DUT	MinimalConfig、单核、VLEN128
原波形构建	EMU_THREADS=1、EMU_TRACE=fst

部署依次完成 WSL 资源调整、固定仓库与递归依赖检出、工具版本核对、NEMU reference 构建、香山仿真器构建、hello/vadd 编译和差分运行。各步保存命令、标准输出、退出码、版本与 SHA。环境配置详情与完整锁文件作为附件交付。

以下是既有脚本的操作顺序，实际操作应在已核对身份的专用输出目录执行。

```
source team/scripts/activate.sh
bash team/scripts/check-environment.sh
bash team/scripts/stage1.sh nemu
bash team/scripts/stage1.sh emu
TEAM_ID= 正式团队标识 bash team/scripts/stage1.sh workload
bash team/scripts/stage1.sh run
```

原始启动曾在 PC 0x10000004 发生寄存器差异。定位发现两端默认 flash 分别含 3 条和 6 条启动指令，随后由同一 boot-flash.S 生成 12 字节镜像，并显式传入共同 flash、128MB RAM 和固定 reference。保存的失败与修复日志可说明启动条件对差分验证的影响。

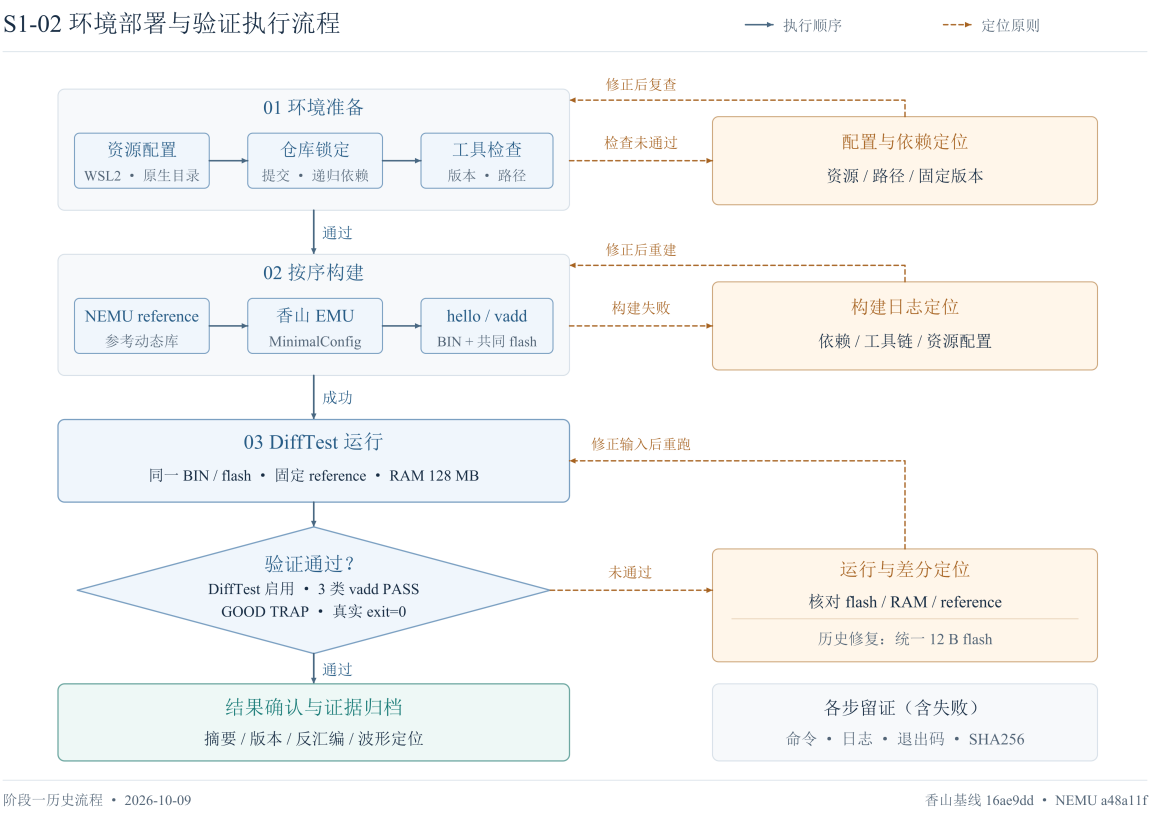


图 1-2　环境部署与验证执行流程

注：流程依次包括环境配置、命令检查、工具核查、NEMU/EMU 与工作负载构建，以及 DiffTest 验证和证据归档；失败时返回相应定位环节。

来源与范围：用户提供示意图；依据：阶段一真实执行顺序、构建和运行日志。版本范围：2026-10-09 阶段一流程。

1.3　hello 与 vadd 验证结果

hello 输出包含团队标识和 WSL IPv4。历史记录使用 riscv-vdot-team，正式录屏应呈现确认后的团队标识及相应真实输出。下文附真实历史输出，正式团队标识由成员确认后补入。

vadd 程序设置 SEW8、LMUL1、VL8，执行两次向量加载、一次 vadd.vv 和一次向量存储，逐字节对比标量参考。

表 1-2　hello 与 vadd 验证结果

用例	输出字节
正数	9 9 9 9 9 9 9 9
负数补码	254 252 250 248 246 244 242 240
边界回绕	128 127 0 0 0 0 254 254

原运行同时具备 HELLO_VADD_PASS cases=3 lanes=24、GOOD TRAP、启用 DiffTest 及 exit0。程序整体统计为 16557 条指令、42962 周期，包含启动、UART 和校验。真实日志摘录见附录。

Historical hello/vadd output; original run 2026-10-09

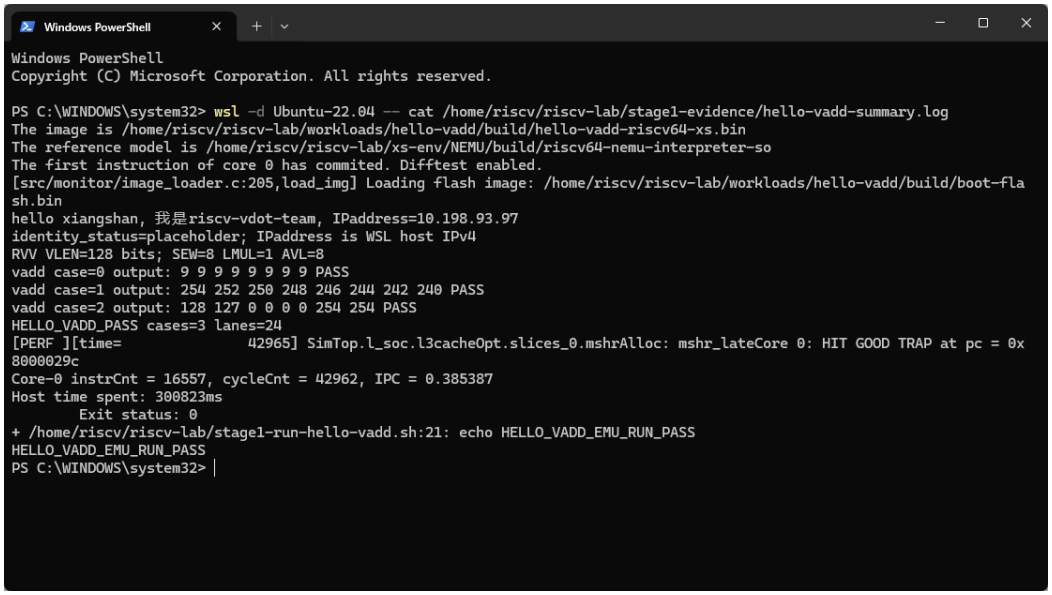


图 1-3 hello 与 vadd 正常结束输出

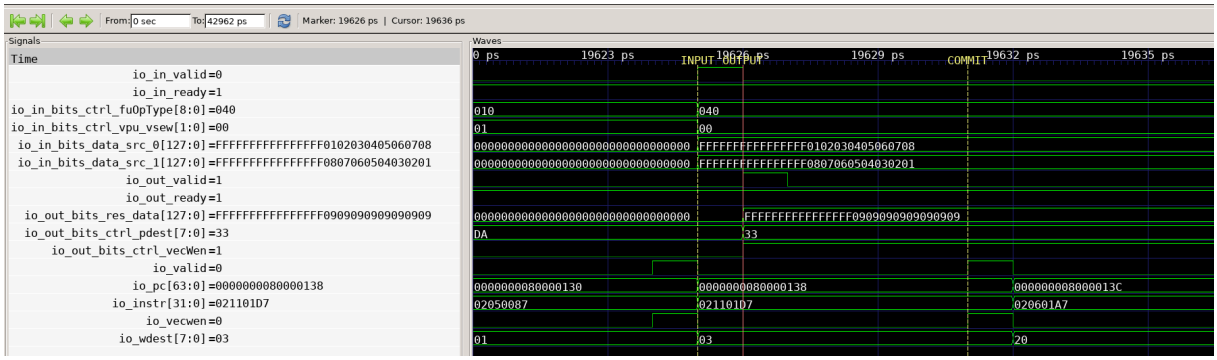
注：已插入用户提供的历史运行日志截图，包含 DiffTest、三类 vadd 输出、GOOD TRAP 及原退出状态。团队名仍为占位标识，正式团队标识输出待补。

来源与范围：用户截图 WindowsTerminal_m5hXeDJnV1.png；原 hello-vadd-summary.log。版本范围：2026-10-09 历史运行回放，不是本次重新运行。

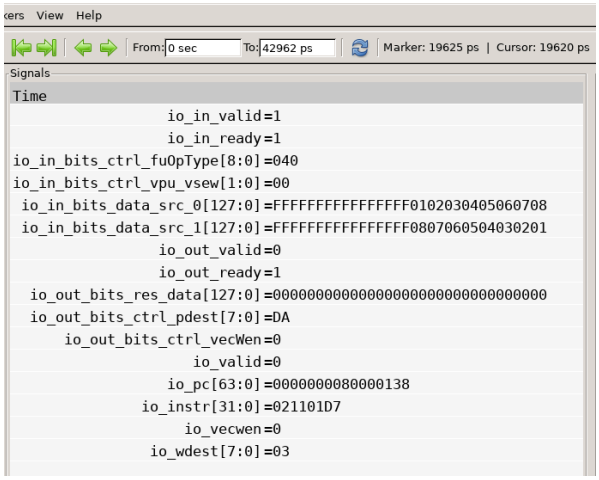
1.4 波形与执行过程

处理器完成取指和译码后，由重命名、调度和操作数准备驱动向量执行。两个 vle8.v 提供输入，vadd.vv 完成按元素加法，结果经写回进入目的向量寄存器并在 ROB 提交。DiffTest 在架构状态层与 NEMU 对应执行结果比较。

(a) Output handshake, marker 19626 ps; all three event positions visible



(b) Input marker 19625 ps



(c) Commit marker 19631 ps

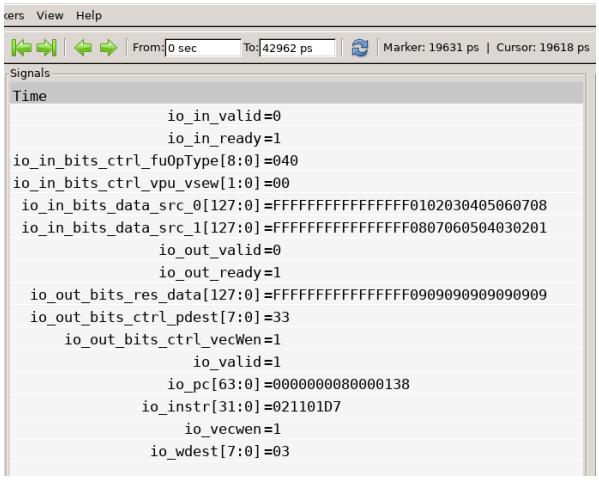


图 1-4　vadd 正数用例的执行与提交波形

注：(a) 输出事件及输入/提交位置；(b) 输入游标处信号值；(c) 提交游标处信号值。截图来自原 FST 提取的 selected VCD，未显示时钟；19625/19626/19631 为文件中事件坐标，截图单位为 ps，不能据此推断芯片物理时钟或实际延迟。

来源与范围：用户上传正数输出截图及 riscv-lab/picture 中的输入、提交截图；保留原始图并仅裁剪外部空白。版本范围：2026-10-09 原基线 vadd；时钟截图仍待补，不是 0f vdot。

原波形提取的 vadd 提交 PC 为 0x80000138，机器码为 0x021101d7，vecwen=1、wdest=3。

表 1-3　波形与执行过程

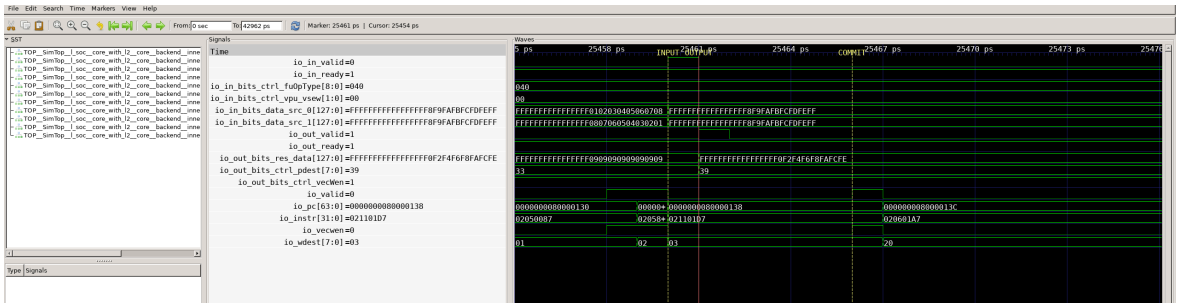
用例	输入事件坐标	输出事件坐标	提交事件坐标	运算输出低 64 位
正数	19625	19626	19631	0x0909090909090909
负数	25460	25461	25466	0xf0f2f4f6f8fafcfe
边界	33009	33010	33015	0xfefe000000007f80

按原波形展开时，应展示时钟、valid/ready、提交 PC 和结果信号，并用游标标注表内事件。具体信号层级以原波形与事件提取脚本为准。表内三列事件数值为采样坐标，结果可说明运算单元输出早于架构提交的流水关系。

完整 FST 位于本机/home/riscv/riscv-lab/stage1-evidence/hello-vadd.fst，大小 220295598

字节。历史波形按 2026 年 10 月 9 日基线标注。最终 0f 环境的 hello/vadd 资格由 R5 组合验收另行绑定。

(a) Negative case: output marker 25461 ps



(b) Boundary case: output marker 33010 ps

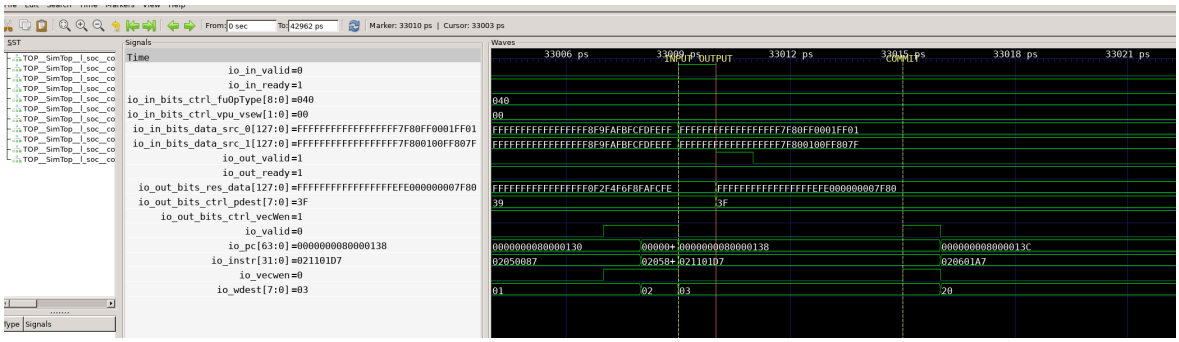


图 1-5　vadd 负数与边界用例结果波形

注：(a) 负数输出低 64 位 F0F2F4F6F8FAFCFE；(b) 边界输出低 64 位 FEFE000000007F80。两图保留各自输入、输出和提交事件位置；未显示时钟，坐标单位按截图为 ps。

来源与范围：用户上传 msrdc_yWBpPeYM8q.png 与 msrdc_6cIzJ9aIVJ.png。版本范围：同一 2026-10-09 历史 vadd 波形；低 64 位按有效 8 个元素解释。

1.5　结论及附件

阶段一已完成部署、共同启动条件、3 类向量加法用例与差分检查。提交报告应附操作日志索引、工具/子模块版本、源文件、反汇编、事件 CSV 及波形定位信息。本稿提供原波形提取表与事件时间线，团队标识待确认。

附件位置为 team/evidence/stage1-20261009，包含 hello-vadd-summary.log、工具版本、子模块锁、boot-flash 反汇编、vadd-commit-events.csv 及 vector-fix-alu-events.csv。运行源码位于 team/workloads/hello-vadd。

阶段一原记录 • 环境记录

1.6　实际执行日志摘录

下列片段直接取自原始归档，日期及路径按原记录保留。完整日志随本文源文件的 evidence 目录提供。

1.6.1 构建和运行时间记录

- 2026 年 10 月 9 日 20:42:15，主动中断并恢复 C++ 构建，将并行度调为 4，已完成对象保留。
- 2026 年 10 月 9 日 20:53:32，EMU 构建结束，状态文件记录 exit_code=0。
- 2026 年 10 月 9 日 20:55:26，workload 与共同 flash 构建结束，记录 WORKLOAD_BUILD_PASS。
- 原正常运行至 21:00:28 结束，退出状态为 0。

1.6.2 程序构建

```
+ /home/riscv/riscv-lab/stage1-build-workload.sh:11: date -Is
2026-10-09T20:55:26+08:00
+ /home/riscv/riscv-lab/stage1-build-workload.sh:12: cd /home/riscv/riscv-lab/workloads/hello-vadd
+ /home/riscv/riscv-lab/stage1-build-workload.sh:13: python3 -
+ /home/riscv/riscv-lab/stage1-build-workload.sh:20: make ARCH=riscv64-xs MARCH=rv64gcv -j1
# Building hello-vadd [riscv64-xs] with AM_HOME {/home/riscv/riscv-lab/xs-env/nexus-am}
+ CC main.c
# Building lib-am [riscv64-xs]
+ AS src/nemu/common/mainargs.S
+ AR -> build/am-riscv64-xs.a
# Building lib-klib [riscv64-xs]
# Creating binary image [riscv64-xs]
+ LD -> build/hello-vadd-riscv64-xs.elf
+ OBJCOPY -> build/hello-vadd-riscv64-xs.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:21: riscv64-linux-gnu-gcc -c -march=rv64gc -mabi=lp64d
boot-flash.S -o build/boot-flash.o
+ /home/riscv/riscv-lab/stage1-build-workload.sh:22: riscv64-linux-gnu-objcopy -O binary -j .text
build/boot-flash.o build/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:23: riscv64-linux-gnu-objdump -d build/boot-flash.o
+ /home/riscv/riscv-lab/stage1-build-workload.sh:24: cp build/boot-flash.bin
/home/riscv/riscv-lab/stage1-evidence/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:25: sha256sum build/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:26: cp build/hello-vadd-riscv64-xs.txt
/home/riscv/riscv-lab/stage1-evidence/hello-vadd-disassembly.txt
+ /home/riscv/riscv-lab/stage1-build-workload.sh:27: sha256sum build/hello-vadd-riscv64-xs.elf
build/hello-vadd-riscv64-xs.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:28: echo WORKLOAD_BUILD_PASS
WORKLOAD_BUILD_PASS
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: rc=0
++ /home/riscv/riscv-lab/stage1-build-workload.sh:1: date -Is
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: printf 'exit_code=%s end=%s\n' 0
2026-10-09T20:55:26+08:00
```

1.6.3 差分运行与结果

```
The image is /home/riscv/riscv-lab/workloads/hello-vadd/build/hello-vadd-riscv64-xs.bin
The reference model is /home/riscv/riscv-lab/xs-env/NEMU/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image:
/home/riscv/riscv-lab/workloads/hello-vadd/build/boot-flash.bin
```

```
hello xiangshan, 我是 riscv-vdot-team, IPaddress=10.198.93.97
identity_status=placeholder; IPaddress is WSL host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
[PERF ][time=              42965] SimTop.l_soc.l3cacheOpt.slices_0.mshrAlloc: mshr_lateCore 0: HIT GOOD
TRAP at pc = 0x8000029c
Core-0 instrCnt = 16557, cycleCnt = 42962, IPC = 0.385387
Host time spent: 300823ms
      Exit status: 0
+ /home/riscv/riscv-lab/stage1-run-hello-vadd.sh:21: echo HELLO_VADD_EMU_RUN_PASS
HELLO_VADD_EMU_RUN_PASS
```

1.7 从原波形提取的事件时间线



图 1-6 vadd 输入、输出与提交事件时间线

注：按 PC 0x80000138、指令 0x021101D7 及目的 v3 精确关联三类 vadd 用例。数字为原 CSV 事件坐标；三列按事件阶段排列，间距不表示时间长度。

来源与范围：evidence/vector-fix-alu-events.csv 及 vadd-commit-events.csv；生成源与哈希见 figures/evidence-20261105。版本范围：2026-10-09 历史 vadd 事件；不能推导物理时钟或新硬件延迟。

图中坐标来自原 FST 提取 CSV，为事件重绘示意。负数和边界用例的对应坐标列于正文表格，完整原波形以 SHA 绑定。

1.8 日志附件索引

evidence 目录保留 NEMU 构建、EMU 构建、workload 构建、原正常运行、原启动失败及并行度调整日志，并包含工具版本、子模块锁、两份事件 CSV 及信号索引。原始日志用于追溯实际操作，本文摘录用于阅读。

1.9 工程实现与验证方法补充

1.9.1 部署任务的依赖关系

环境部署包含宿主运行时、硬件生成工具、RISC-V 程序工具链和参考模型四条依赖链。Java 与 Mill 负责解析 Scala 和 Chisel 工程，firtool 将中间表示转换为可仿真的硬件描述，Verilator 与宿主 C++ 编译器生成 EMU。RISC-V GCC 和 binutils 处理目标程序及启动镜像。NEMU 在参考侧执行同一条架构指令，为差分检查提供状态。

四条依赖最终在一次真实运行中汇合。若只验证各工具能够打印版本，仍无法确认其组合产物相互兼容。因此阶段一把工具检查、实际构建、共同启动镜像、程序执行及差分结果串成完整链路。各层分别保存输入和输出摘要，使后续故障能够定位到对应环节。

表 1-4 部署任务的依赖关系

部署对象	上游依赖	主要产物	核对内容
NEMU 参考库	固定源码、配置、宿主编译器	动态参考库	配置、编译退出码、加载路径、SHA
香山 EMU	固定硬件与子模块、Java/Mill、firtool、Verilator	宿主仿真器	DUT 参数、生成命令、编译器、SHA
裸机程序	nexus-am、klib、RISC-V 工具链	ELF/BIN、反汇编	ISA/ABI、入口地址、段、机器字
共同 flash	启动汇编与 binutils	12 字节启动镜像	两端加载同一实物
差分运行	上述四类产物	原始日志、退出状态、波形	完整通过标记及实际状态比较

1.9.2 内存、swap 与并发的工程含义

香山仿真器构建会同时消耗 Java 堆、硬件生成工具内存、多个 C++ 编译进程及文件缓存。仅提高 Java 堆并不能保证构建更快，反而可能挤压宿主和其他阶段可用内存。本项目在约 27.7GiB 物理内存的个人电脑上，将 WSL 上限配置为 20GB，并设置 16GB swap 作为资源缓冲。

并发度需要结合当前阶段选择。原始记录中将 C++ 构建调整到-j4，并明确保留已完成对象。日常复现脚本采用较低默认并发，以便控制内存压力。两种配置的具体发生时间和真实命令分别记录，报告只据此说明实际操作。运行阶段每个 DUT 仍按单核、单仿真线程进行，宿主同时管理多个任务的数量另按对应合同约束。

swap 发生时，墙钟耗时可能显著增加。构建速度和模拟吞吐用于安排工作，性能表

中的 cycles 由处理器仿真内部计数器给出。对于较长作业，还应保存内存峰值、日志大小、单调时钟限额和进程回收结果，以便判断完成状态。

1.9.3 固定源码与依赖恢复

xs-env 协调硬件、NEMU、nexus-am 和 DRAMsim3 等依赖。项目记录每个组件的完整提交，而不是只记录分支名。相同分支在后续日期可能移动，固定提交则可以明确回答当时使用了哪一份代码。

恢复环境时先读取版本锁，再检查已有目录的提交与修改状态。遇到不一致时应保留现场并使用隔离目录，避免覆盖旧实验。递归子模块的身份同样属于硬件输入，单独更新主仓库并不能保证恢复了同一设计。

新成员需要区别三种版本信息。第一种是香山上游基线，说明开发起点。第二种是包含 vdot 修改的硬件提交，说明最终设计。第三种是后续文档与封存提交，说明交付材料。三者共同构成可追溯记录，后两者之间必须通过源码树和产物摘要连接。

1.9.4 程序装载、启动与串口输出

目标程序以 ELF 承载符号、段信息和调试资料，以 BIN 供加载器装入。反汇编用于核对实际机器字和指令地址。即使 C 源代码未变，链接参数、数据布局或库版本发生变化，也可能改变指令 PC，因此观察波形时应使用与被测 BIN 同次构建的反汇编。

hello 输出先确认程序已经经过启动代码到达 C 入口，并检查 UART 可见结果。随后程序启用并核对向量状态，执行三类 vadd 用例。输入数据、标量期望和完整输出比较均由程序实际执行，输出 PASS 之前已完成逐元素检查。

原 hello 包含 WSL 宿主 IPv4，该地址用于识别当时开发环境。它与仿真处理器中的网络设备能力分别表述。团队标识目前仍使用历史占位符，填写正式队名后应保留新镜像和输出的身份，原历史记录继续作为原实验凭据。

1.9.5 启动差分故障的定位过程

原故障发生在启动阶段 PC 0x10000004，程序尚未进入用户向量运算。检查差分首次不一致的位置后，进一步比较两端启动 flash，发现默认启动指令数量不同。若直接调整测试期望，无法解决两端起始状态和执行序列的根本差异。

最终处理使用一份 boot-flash.S 构建共同镜像，并在启动命令中显式指定该文件。修复之后，两端经过相同启动序列，再加载同一程序，在保持 DiffTest 启用的条件下完成验证。该过程说明最早的差分错误应先从共同输入、复位和装载条件检查。

表 1-5 启动差分故障的定位过程

排查步骤	所查证据	得到的判断
定位首次差分	原失败日志、PC、整数状态	发生在启动代码
比较输入	默认 flash 及反汇编	两端默认序列不一致
统一启动实物	共同 boot-flash.bin	输入条件明确
使用原比较域恢复运行	正常输出、GOOD TRAP、退出状态	hello/vadd 通过



图 1-7 共同启动镜像修复前后的差分定位

注：首次差异在启动 PC 0x10000004、t0，原 instrCnt 为 2；随后两端通过显式 flash 参数使用同一 12 字节文件，三条指令按原文件偏移列出。恢复日志记录三类 24 lane 通过、GOOD TRAP 及 exit 0。

来源与范围：2026-10-09 原失败日志行 164/166、boot-flash-disassembly.txt、完整成功日志行 7/10/16/17/19 及 hello-vadd-summary.log；完整出处和哈希见图数据附件。版本范围：历史启动问题定位；不是 vdot 错误或新运行。共用文件不等于两端物理映射地址相同。

1.9.6 向量加法结果的解释

SEW8 下每个元素保存 8 位加法结果，运算按模 256 保留低 8 位。负数补码的无符号打印值应结合原输入解释。例如十进制 254 对应 8 位补码 −2，打印成 254 不表示数学

结果变为正数。边界用例专门包含回绕，以检查程序的参考计算与向量指令行为一致。

结果进入执行单元输出后，仍需经过写回和乱序提交。本项目把输入事件、输出事件和 ROB 提交分别标出。正数用例的输入 19625、输出 19626、提交 19631 说明三个事件属于不同流水位置。表中单次差值用于解释该次观测，不据此给出所有操作的固定提交延迟。

1.9.7 日志附件的阅读和复查顺序

建议先查看退出状态和通过摘要，确定记录是否完整。再沿日志开头的镜像、reference 与 flash 路径核对输入。随后检查反汇编和事件 CSV，最后在需要时打开完整波形。这样可以避免仅凭局部 PASS 文本就扩大结论，也能减少在大波形中盲目搜索的时间。

本报告将精简日志放在正文，原始构建和运行日志作为附件保存。构建日志中的工具调用与运行日志中的架构输出承担不同用途，不能互相替代。附后的工具版本和子模块锁用于重建依赖身份，完整 FST 按 SHA 定位，便于他人核对真实信号。

1.9.8 环境复现的输入层级与检查顺序

复现阶段一时，首先确认工作树与子模块的源码身份，其次确认工具链身份，最后检查运行输入。这个次序对应由源到产物的因果关系：源码决定应当构建什么，工具决定怎样生成，镜像和启动参数决定实际执行什么。仅保存最终屏幕输出，无法区分输入变化与工具变化；仅保存版本号，也无法证明运行确实加载了固定产物。报告把这些信息分别列出，是为了让后来者能够从差异发生的最早环节向下排查。

同名目录不能代表同一环境。一个已经存在的 xs-env 目录可能包含未提交修改，子模块也可能停在另一提交。复现者应在启动构建前记录主仓库及递归子模块的完整提交，并检查脏文件；若发现与报告锁定值不同，应先留存差异，再在隔离目录恢复。此规则同时保护旧实验，因为直接覆盖原目录会使后来者无法重读当时的失败证据。

工具版本清单也需要分清直接工具与间接依赖。Java、Mill、firtool、Verilator 和宿主编译器处于硬件生成链；RISC-V GCC、汇编器和链接器处于目标程序链；NEMU 的构建还依赖宿主编译工具。某一工具升级后，即使硬件源文件不变，生成 RTL、C++ 仿真器或 ELF 仍可能变化。阶段一并不凭工具版本推断语义改变，而是在复验时将版本、命令、产物摘要和运行结果一起核对。

当构建失败时，错误发生的位置比最后一行提示更重要。Java 依赖解析失败意味着尚未进入硬件生成；firtool 失败意味着无法获得可供 Verilator 使用的目标 RTL；C++ 编译失败则需要区分缺头文件、资源不足和实际源错误。每次修复后只重新执行受影响的下游步骤，同时保留失败日志与原命令。这样可以解释为什么最终日志出现成功，并防止把一次不同配置的成功结果误接到失败尝试后面。

运行所需的镜像还存在不同层级。ELF 提供符号、段布局与反汇编线索，BIN 是加载器使用的内容，flash 是启动入口的独立输入。研究一次具体提交事件时，应先用 BIN

和 ELF 确认机器码，再用实际启动命令确认两侧加载了哪些文件。文件名可以被复用，SHA 和文件长度才可以约束具体实物；入口地址和装载位置则约束这些字节进入处理器的方式。

阶段一的验收采用递进关系。环境检查通过只说明依赖齐备，构建通过说明产物已生成，hello 成功说明程序和串口链路可运行，vadd 逐元素成功说明这组向量输入得到了预期结果，DiffTest 正常结束才为该运行增加参考模型一致性证据。各级结论不能倒置。例如看到 GOOD TRAP 但没有找到输出比较或真实退出状态，仍不足以断言向量功能已被检查；看到某一工具版本，也不能推导完整 EMU 已经重建成功。

1.9.9 启动一致性问题的反事实分析

启动处首次差分位于 PC 0x10000004，这一位置早于用户测试代码。因此排查优先级从指令算法转向复位状态、flash 内容与装载配置。复核时同时核对该地址在两侧实际读取的机器字，并用对应镜像的反汇编确认指令；地址本身不足以证明两次运行使用了相同内容。若在此处修改 vadd 预期值，错误仍会在进入 vadd 以前发生；若关闭 DiffTest，程序可能继续运行，却失去原本用于识别启动差异的门禁。保留原失败、机器字及镜像身份，使修复原因可以被独立检查。

比较两侧默认启动内容后，项目选择共同 boot-flash.S 生成一份明确实物，再将该实物传入运行。此时“共同”有两个含义：字节身份相同，且加载参数指向相同实物。单纯在源码中写出相同汇编，仍可能因工具或构建时点不同得到不同二进制；只传相同路径，仍可能因目录映射或文件被覆盖而不一致。因此日志中同时记录生成命令、输入摘要和运行参数。启动脚本或镜像更新后，应重新核对地址到机器字的映射，不能沿用旧反汇编。

修复后的正常运行构成因果链中的最后一环。两侧先经过共同 flash，再到达同一程序入口，随后观察 hello、向量用例与差分终态。失败和成功是两个独立运行，报告没有把前者的局部现象说成后者已经通过，也没有把后者覆盖前者。未来若更换启动镜像或参考库，应重新核对首次差分位置与终态，而不能沿用本次的成功标签。

1.9.10 向量用例如何支持结论

三个 vadd 用例承担不同角色。正数输入便于检查基本取数、计算和写回；负数输入检查补码解释以及打印形式与算术含义的区别；边界输入检查定宽结果的回绕。用例组合的目的不是穷举全部向量 ISA，而是在自定义指令开发之前证明共同仿真链路可以观察到向量运算的关键状态，并能对典型错误给出可辨结果。

参考计算应与硬件结果采用相同的元素宽度和截断规则。SEW8 的加法输出只保留低 8 位，一个数学和超过 255 时会按模 256 回绕；负数以补码存储，若按无符号十进制打印，就会得到大于 127 的数。报告保留输入、期望和实际输出，而不只列 PASS，是为了让读者能够在不用重新运行仿真的条件下核对这一解释是否成立。

波形事件表提供的是一次实际执行的定位点，而不是固定微架构时延合同。输入、输出和提交落在不同拍，说明指令在执行单元内完成后，还需要经过父级选择、写回及 ROB 退休。由于乱序状态、下游 ready 和周边指令可以改变这些间隔，不能从一次正数用例的三个事件坐标推导所有 vadd 的固定延迟。报告同时给出 PC 和事件类型，方便将波形与反汇编、运行日志连接。

波形截图和 CSV 的证据能力也不同。截图适合展示某几个关键信号在局部时间范围内的相对位置；CSV 可用于逐事件复算和脚本检查；完整 FST 保留更广的信号与时间线。若截图缺少时钟，仍可以说明它来自原波形数据，但不能据坐标差推算芯片物理时间或频率。阶段一明确保留待补时钟图位，避免用一个没有时钟的局部截图替代时序证据。

1.9.11 构建资源与失败证据的管理

硬件构建耗时较长，且 C++ 编译能够产生大量并行任务。出现资源压力时，先记录实际内存限制、swap 配置和并行度，再观察进程退出码与构建日志。若只提高并行度，可能使多个编译进程同时争抢内存；若只增加 swap，可能使作业从失败变为极慢完成。报告中的资源配置是该主机上的实际条件，不推广为每台电脑都应采用的固定值。

Windows 工作树与 WSL 原生目录的文件访问特征不同，仓库规模又较大。Git 索引、硬件生成和大规模 C++ 编译应各自选择有证据支持的放置位置，不能因为某一次超时就给所有操作规定一个加速比例。若跨系统复制源码，必须重新核对源码快照和必要文件摘要，并隔离可写构建目录。这样可以保持历史实验可读，也能让新的构建身份独立于旧缓存。

失败日志不是应当清除的噪声。它们解释为什么后续命令改变了参数、并发或启动镜像，也帮助识别一个看似成功的结果是否来自另一轮运行。每项日志至少应有任务、时间、命令、退出码和所用输入的对应关系。对后续阶段来说，保留失败位置尤其重要，因为新 vdot 的差分失败可能来自指令语义，也可能再次来自启动或参考模型装载；阶段一的排查记录给出了可重复的分层方法。

1.9.12 阶段一成果向后续阶段的传递

阶段一交付的核心不是某个固定波形画面，而是已验证的运行入口和输入身份。阶段二可在该入口上增加 vdot 译码、算术与父级控制检查；阶段三可以继续使用共同镜像和差分终态检查，同时增加自定义指令参考模型。每次增加新能力都要说明哪些旧证据仍适用，哪些输入和源码变化要求重测。这样的传递规则避免将早期 hello/vadd 通过误写成新 vdot 或新性能候选已经通过。

对后来复核者，最短路径是先打开阶段一版本表与最终运行摘要，核对镜像、flash、reference 和 EMU 的身份，再查看三组向量输入及期望，最后针对疑点深入日志与波形。若摘要与原始日志不一致，应以原始运行事实和退出状态为准，暂停扩大结论。这个阅

读顺序使报告既能快速说明已完成的基础验证，也能支持对关键环节的独立复查。

1.9.13 故障定位时的观察边界

在仿真系统中，“没有看到预期输出”可以对应许多不同情况：构建没有完成、程序没有加载、启动未到达 C 入口、UART 尚未刷出、参考模型提前拒绝，或者真正的算术结果错误。报告因此把观察边界分为构建完成、启动到达、指令执行、架构比较和进程结束五个环节。每个环节都有可核对的证据，不能只用一个终端窗口的最后几行去解释全部过程。

构建完成的边界由工具退出码和产物检查给出。进程结束的边界由 EMU 返回值、GOOD TRAP 和差分总结共同给出。两者之间仍有大量可能的状态，因此启动日志中的 PC、flash 路径、装载地址和镜像摘要不能省略。若程序实际加载的 ELF 与反汇编来自不同构建，研究者可能把正确的事件错误地归到另一条指令；若终端输出被截断，也可能把未完成任务误当成正常通过。

观察点还要区分架构状态与微架构状态。UART 文本是软件可见输出，向量寄存器和 CSR 是架构状态，ready、valid、ROB 标签和内部时钟是实现状态。阶段一的 vadd 结果主要支持架构功能判断，波形中的握手与提交点帮助解释发生过程，却不能单独代替完整架构比较。到了 vdot 阶段，必须继续把这两类状态分别验证，尤其不能因内部 result 出现正确数值，就跳过完整写回与提交检查。

即使两个观测源给出相同的时间数字，仍需确认单位和事件坐标。FST/VCD 中的时间刻度、仿真器周期数和主机墙钟属于不同量。日志打印顺序也可能受缓冲影响，不能直接当成硬件边沿顺序。阶段一将事件 CSV 绑定到原波形和实际运行，说明某个信号在给定时间坐标发生变化；关于性能或时序的结论，则要等后续阶段建立专门的计数窗口与综合合同。

1.9.14 可重现结果与不可推广结果

本轮环境验证在固定主机、固定工具和固定输入下完成。复现成功支持这些条件下的可执行性，但不会自动证明另一操作系统、另一子模块提交或另一工艺库也能得到相同输出。报告列出本机内存、WSL 配置和路径，是为了复核实际条件；新机器可以采用等效配置，却必须重新核对产物和运行结果。

hello 和三组 vadd 说明基本链路已经闭合，但测试范围依旧有限。它们没有覆盖完整 RVV 指令集，也没有提供 vdot 自定义编码的通过证据。特别是 vadd 的数学和写回宽度与后续 vdot 不同，不能把阶段一的负数或边界用例视为 vdot 极值测试。将结论限制在实际执行的程序和观察域内，是后续阶段继续扩展测试的前提。

对于启动故障，统一 boot flash 证明本次失败原因得以排除，但并不表示启动流程永远不会再次变化。若更新 EMU、NEMU 或加载脚本，仍要检验两侧是否加载同一启动实物。若将来重新编译工作负载，也要使用新镜像对应的反汇编和 PC 表。报告保留

这些条件，避免“曾经通过”成为忽略输入变化的理由。

原始证据中的历史日期和主机路径不会为了当前文档外观而改写。材料日期描述报告整理时点，日志日期描述实验执行时点，二者承担不同身份。后续若团队补充正式截图、署名或报告封面，应将新增内容标为新的材料修订，并保留原始记录与摘要关联。这样既能改善交付可读性，也不会破坏对当时实验的追溯。

1.9.15 对阶段二验证设计的具体启示

阶段一确认了程序能够启动、向量配置能够进入执行路径、差分系统能够观察架构结果。阶段二增加 vdot 时，可沿用同一分层原则设计用例：先用独立软件模型核查数值，再用局部 RTL 检查位宽和握手，随后在真实父级核查取消、反压与时钟，最后在全核检查译码、配置依赖、异常和 128 位写回。每一层都有独立目标，出现失败时能够回退到更小的复现范围。

阶段一的正数、负数和边界三类组织方式可以借鉴，但 vdot 需要新的极值组合。点积有八项乘法与求和，乘积符号和归约位宽比单元素加法复杂；完整向量结果的高位还必须按自定义指令合同处理。一个 vadd 用例可以证明环境能够运行向量程序，却不能替代这些新语义检查。报告在此明确两阶段的证据传递边界，减少重复部署的同时保留新功能的独立验收。

测试入口应允许对失败状态进行负控。阶段一已有启动差异这一真实失败，说明比较机制能在错误起点停止；阶段二还需要对算术位、写回高位、身份和取消行为分别注入或构造错误，证明检查器会拒绝。只有正常通过而没有拒错能力说明，某些不被观察的错误可能仍然混入结果。负控的对象和期望失败位置必须明确，不能把预期失败记作设计失败。

最后，构建与运行成本不应混入指令性能。阶段一的墙钟主要反映源码生成、C++ 编译、模拟执行和宿主资源；后续性能报告使用处理器内部周期与退休计数，并要求同镜像、同工作量配对。即使某台机器编译得更快，也不会改变同一 EMU 中已记录的架构周期。把这两种数字分开陈述，才能让环境改进和硬件优化各自得到正确评价。

1.9.16 阶段一验收记录的阅读方式

阶段一的交付物同时包含环境部署记录、失败定位过程和成功运行日志。审阅时应先确认每份日志对应的源码、脚本与镜像，再确认运行命令及退出状态，最后检查程序自己的通过标记。若只阅读汇总中的“成功”二字，可能遗漏中途替换过的 boot flash 或某次构建使用的旧产物。报告保留失败前后两组证据，是为了让修复因果链可以复算，而不是把失败文件当作无关噪声删除。

对于向量用例，还要区分输入覆盖和观察覆盖。输入覆盖说明测试提供了哪些带符号数、边界值和寄存器初态；观察覆盖说明实际比较了哪些向量元素、异常字段和终止状态。输入看起来复杂并不保证所有结果都被断言，单条通过日志也不保证其余向量状

态没有变化。每个用例应能从初态推导预期，并在日志或检查器中找到对应的比较点。若缺少观察点，只能报告“执行过”，不能报告“验证过”。

复现实验时，最容易被忽略的是运行输入和编译输入之间的版本断层。源码更新后若复用旧 EMU，现象可能与新代码无关；镜像更新后若沿用旧反汇编，PC 定位也会发生偏差。阶段一把可执行文件、启动镜像、程序镜像和反汇编视为一组关联产物，并记录生成顺序。新复现者可以使用不同的绝对路径，但需要在同一轮运行中重新确认这组产物的关联性。

阶段一最终能够支持的结论是：在已记录环境与固定输入下，启动差异得到解释和修正，基本程序及选定向量用例完成验证，后续自定义指令开发具备可重复的运行入口。这个结论不包含新点积指令的语义正确性，也不包含后续父级门控与性能指标。明确边界让阶段二能以现成环境为基础，继续添加真正针对点积的模型、协议和全核测试。

1.10 环境部署详细方法

本节整理固定项目材料中的详细方法和原始合同。设计阶段的“拟实现”“必须验证”等表述属于原合同约束，当前完成状态以本报告正文和 R5/P5 证据为准。原日期、比较合同及未测边界按来源保留。

来源：team/docs/environment.md

1.10.1 Windows/WSL 原生文件系统与工具分工（工程实践）

此处是工程执行的默认选择，不改变任务包、验证合同或资源预算，也不把原生工具选择本身视为功能、性能或 T17 验收。

表 1-6 Windows/WSL 原生文件系统与工具分工（工程实践）

源码 / 工作树和任务	默认执行位置	注意事项
Windows NTFS（如 D:\...）工作树上的 Git status/diff、完整索引检查、commit/push	**Windows 原生 Git**，直接使用 Windows 路径	避免经 /mnt/d 跨文件系统遍历大型 Git 工作树；发布仍遵守 Issue 短集成窗口、远端核对和快进规则
Linux/WSL 原生目录（如 /home/...）中的 Git	**Linux 原生 Git**	不因“统一用 Windows Git”而倒置跨文件系统访问
Chisel/Scala、硬件生成、RISC-V 工具链编译、EMU/NEMU、波形 / 综合时序等重型任务	**Linux/WSL 原生文件系统**，使用 Linux 工具	以当前工作包的固定工具和真实硬件 / 软件输入为准；受总体 CPU、内存和其他 Agent 的资源保护约束

表 1-6 Windows/WSL 原生文件系统与工具分工（工程实践）（续）

源码 / 工作树和任务	默认执行位置	注意事项
Windows 工作树与 Linux 运行目录之间的源码、二进制及证据交换	独立检出或固定快照，并在目标环境核验	记录完整 HEAD、未提交差异 / 源码快照、外部依赖和关键文件 SHA-256；不可直接把另一个工作树的 PASS 当作本次 PASS

执行前先确认实际仓库根、工作树位置、分支、完整 HEAD、Git common-dir、未提交改动和正在使用的输入 / 输出；选择**与工作树所在文件系统一致**的 Git。需要 Linux 构建时在原生目录准备独立源码与输出，核对身份后执行已有合同；不要仅为省一次复制就反复扫描 Windows 挂载目录。回写报告、准备提交时由目标 Windows 工作树使用原生 Git 检查实际落盘字节与索引，必要时按实验合同核对 Linux 运行快照。多个 Agent 不共用可写工作树或 Git index，既有运行和冻结证据保持受保护。

来源与限制： 2026-10-24 R4 调用入口发布前，一次从 WSL 跨到 Windows 工作树的最终检查在约 300 秒超时，确认未提交、无残留进程和索引锁后，使用 Windows 原生 Git 完成剩余检查与提交（原始发布检查、Issue 记录）。两次执行并非严格的同命令 A/B 基准，不能据此宣称任意 Git 操作有固定加速比例。若某任务实测另一种放置更合理，可保留证据后调整；正确性与可复现性优先。

1.10.2 WSL 内存

在 Windows 的%UserProfile%\.wslconfig 中设置：

```
[wsl2]
memory=20GB
swap=16GB
```

保存正在进行的 WSL 工作后，在 PowerShell 执行 wsl –shutdown，重新进入 Ubuntu，用 free -h 核对。原机器 swap 文件放在 D:\WSL\wsl-swap.vhdx，新成员无需照搬该路径。较小内存的机器相应降低 JVM 堆和构建并行度，不能同时把 JVM 堆和构建进程占满可用内存。

1.10.3 固定源码

表 1-7 固定源码

组件	已验证提交
xs-env	b0b506481e839a5df8a580e76f5aa246dc60fe7c
香山硬件基线	16ae9ddcda54fc9a2fddffad73174cf793ac7814
NEMU	a48a11f208ff81ad765591d83271d1b339fb3861
nexus-am	086a238062fdfb3281b745a537b361983d0154b5
DRAMsim3	1d2a9bf6da8e050d975ef2669c42faa93943f489
DiffTest	62e4540c03c03aa947bdb67f4b6bf66f1a29c5d0
yunsuan	cadd3c2f43096e253d61296b33ac697be8354e29

在团队仓库运行 bash team/scripts/prepare-dependencies.sh：固定检出 xs-env，仅初始化配套 NEMU、nexus-am、DRAMsim3 及其递归子模块，然后锁定 NEMU 额外依赖。团队仓库自己的香山子模块通过 git clone --recurse-submodules 获取。脚本遇到已有依赖版本不同或已跟踪文件修改会停止，不丢弃队友修改。

NEMU 额外依赖的实际提交见锁定脚本。SimPoint 是 NEMU 已记录的子模块，通过递归初始化固定；其他额外下载仓库不能依赖当天默认分支，构建前必须执行锁定脚本。

1.10.4 已验证工具

表 1-8 已验证工具

工具	版本
Oracle GraalVM JDK / javac	21.0.12
LLVM / Clang	19.1.7
Mill 项目版本	0.12.3
Scala / Chisel	2.13.15 / 6.6.0，由项目解析
firtool	1.62.1，由本次项目依赖解析
Verilator	5.052，提交 ea338be98e1e838d3518809ce8899f85a009963c
RISC-V GCC	11.4.0，riscv64-linux-gnu- 前缀
GNU binutils	2.38
宿主 GCC / G++	11.4.0

表 1-8 已验证工具（续）

工具	版本
Make / CMake / Ninja	4.3 / 3.22.1 / 1.10.1
Python / GTKWave	3.10.12 / 3.3.104

完整工具输出见 tool-versions.txt。Ubuntu 软件包版本清单提供历史来源信息，不建议无差别恢复整台主机的软件包。

1.10.5 新机器安装

先完成配套源码检出，然后加载路径。在团队仓库根目录执行：

```
source team/scripts/activate.sh
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target base
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target llvm
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target mill
sudo apt-get install -y cmake ninja-build gtkwave
```

以上安装入口来自固定 xs-env 的 setup-tools.sh。软件源提供的补丁版本可能变化，因此安装后必须运行团队环境检查。

JDK 原安装脚本使用 Oracle 21/latest URL；它会随时间变化，不能据此保证重装仍是 21.0.12。本队提供固定 GraalVM 安装脚本，使用已核对可下载的官方 21.0.12 Linux x64 归档和官方 SHA-256，安装到用户目录，不需要 sudo：

```
export JAVA_HOME="$HOME/.local/share/graalvm-jdk-21.0.12"
bash team/scripts/install-graalvm.sh
source team/scripts/activate.sh
```

新终端中继续导出相同的 JAVA_HOME；已安装原本机 /opt/graalvm-jdk-21 的成员可使用默认路径，无需重装。脚本遇到已有 JDK 版本不同会停止，不能覆盖现有安装。新版本需要单独验证后更新团队记录。安装方法参考 Oracle GraalVM 安装说明；固定归档为 21.0.12 Linux x64，校验来源为 官方 SHA-256。

Verilator 在用户目录源码编译，避免官方脚本默认 -j$(nproc) 导致个人电脑资源不足。先安装依赖，再使用已安装的 Clang：

```
sudo apt-get install -y help2man perl autoconf flex bison libgoogle-perftools-dev \
  libjemalloc-dev numactl perl-doc zlib1g-dev liblz4-dev libfl2 libfl-dev
mkdir -p "$HOME/riscv-tools"
git clone https://github.com/verilator/verilator.git "$HOME/riscv-tools/verilator"
cd "$HOME/riscv-tools/verilator"
git checkout ea338be98e1e838d3518809ce8899f85a009963c
unset VERILATOR_ROOT
autoconf
./configure CC=/usr/local/llvm/bin/clang CXX=/usr/local/llvm/bin/clang++ LINK=/usr/local/llvm/bin/clang++
make -j2
sudo make install
```

回到团队仓库根目录：

```
source team/scripts/activate.sh
bash team/scripts/check-environment.sh
```

此检查核对配套源码提交、香山递归子模块与关键工具版本；香山 HEAD 可以包含本队新增文档提交，但必须保留硬件基线祖先。检查通过不等于新硬件改动已通过仿真。不要手动升级全局 Scala；项目 Mill 会读取仓库.mill-version，启动器来源版本不等于项目版本。

1.10.6 使用现有环境

现有主机只需从 /home/riscv/riscv-lab/xs-env/XiangShan 运行脚本。activate.sh 会设置当前团队仓库为 NOOP_HOME，按 TEAM_XS_ENV_ROOT 指定 NEMU 和 AM，设置 LINUX_GNU_TOOLCHAIN=1。AM 默认不会自动启用向量 ISA，因此构建 workload 仍需显式设置 MARCH=rv64gcv。

1.11 NEMU 构建原始日志

以下为对应历史记录的原始输出，日期、路径和提交身份保留原值。

```
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:9: date -Is
2026-10-09T20:24:27+08:00
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:10: cd /home/riscv/riscv-lab/xs-env/NEMU
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:11: git rev-parse HEAD
a48a11f208ff81ad765591d83271d1b339fb3861
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:12: make riscv64-xs-ref_defconfig
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/nanopb'...
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/LibCheckpoint'...
Submodule 'resource/nanopb' (https://github.com/nanopb/nanopb) registered for path 'resource/nanopb'
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/LibCheckpoint/resource/nanopb'...
Submodule path 'resource/nanopb': checked out '671672b4d7994a9b07a307ae654885c7202ae886'
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/gcpt_restore'...
/home/riscv/riscv-lab/xs-env/NEMU/scripts/config.mk:20: Warning: .config does not exists!
/home/riscv/riscv-lab/xs-env/NEMU/scripts/config.mk:21: To build the project, first run 'make menuconfig'.
+ CC confdata.c
+ CC expr.c
+ CC preprocess.c
+ CC symbol.c
+ CC util.c
+ YACC build/parser.tab.h
+ LEX build/lexer.lex.c
+ CC build/lexer.lex.c
+ CC build/parser.tab.c
+ CC conf.c
+ g++ /home/riscv/riscv-lab/xs-env/NEMU/tools/kconfig/build/conf
+ CC fixdep.c
+ g++ /home/riscv/riscv-lab/xs-env/NEMU/tools/fixdep/build/fixdep
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:13: /usr/bin/time -v make -j1
Makefile:40: You have set CONFIG_FLASH_IMG_PATH option, but the specified path """ does not exist!
```

```
Cloning into 'resource/softfloat/repo'...
+ CC src/nemu-main.c
+ CC src/device/device.c
+ CC src/device/alarm.c
+ CC src/device/intr.c
+ CC src/device/flash.c
+ CC src/cpu/tcache.c
+ CC src/cpu/cpu-exec.c
+ CC src/cpu/instr_stat.c
+ CC src/cpu/difftest/ame/amuctrl.c
+ CC src/cpu/difftest/ame/msync.c
+ CC src/cpu/difftest/dut.c
+ CC src/cpu/difftest/ref.c
+ CC src/monitor/ui.c
+ CC src/monitor/watchpoint.c
+ CC src/monitor/monitor.c
+ CC src/monitor/fdt_rng_seed.c
+ CC src/monitor/aligncheck.c
+ CC src/monitor/image_loader.c
+ CC src/utils/state.c
+ CC src/utils/iqueue.c
+ CC src/utils/br_log.c
+ CC src/utils/expr.c
+ CC src/utils/log.c
+ CC src/utils/filename.c
+ CC src/utils/dynamic-config.c
+ CC src/utils/timer.c
+ CC src/memory/host-tlb.c
+ CC src/memory/vaddr.c
+ CC src/memory/paddr.c
+ CC src/isa/riscv64/system/mmu.c
+ CC src/isa/riscv64/system/priv.c
+ CC src/isa/riscv64/system/trigger.c
+ CC src/isa/riscv64/system/intr.c
+ CC src/isa/riscv64/system/timer.c
+ CC src/isa/riscv64/system/aia.c
+ CC src/isa/riscv64/reg.c
+ CC src/isa/riscv64/ame/raw_fp4_memory.c
+ CC src/isa/riscv64/clint.c
+ CC src/isa/riscv64/difftest/dut.c
+ CC src/isa/riscv64/difftest/ref.c
+ CC src/isa/riscv64/init.c
+ CC src/isa/riscv64/logo.c
+ CC src/isa/riscv64/instr/priv/system.c
+ CC src/isa/riscv64/instr/fp.c
+ CC src/isa/riscv64/instr/rvk/aes_common.c
+ CC src/isa/riscv64/instr/rvk/crypto_impl.c
+ CC src/isa/riscv64/instr/ame/mcompute_impl.c
+ CC src/isa/riscv64/instr/ame/mldst_transpose_fast.c
+ CC src/isa/riscv64/instr/ame/mreg.c
+ CC src/isa/riscv64/instr/ame/mldst_impl.c
+ CC src/isa/riscv64/instr/ame/mldst_fast.c
+ CC src/isa/riscv64/instr/decode.c
+ CC src/isa/riscv64/instr/rva/amo.c
+ CC src/isa/riscv64/instr/rvv/vldst_impl.c
+ CC src/isa/riscv64/instr/rvv/vfexp2.c
+ CC src/isa/riscv64/instr/rvv/vcompute_impl.c
+ CC src/isa/riscv64/instr/rvv/vreg_impl.c
+ CC src/isa/riscv64/instr/rvv/vcommon.c
```

```
+ CC src/isa/riscv64/instr/rvcbo/cbo_impl.c
+ CC src/isa/riscv64/instr/mp.c
+ CC src/isa/riscv64/instr/rvb/rvintrin.c
+ CC src/isa/riscv64/instr/rvt/sfu.c
+ CC src/isa/riscv64/instr/rvt/matmul_subfloat.c
+ CC src/isa/riscv64/instr/rvt/element_wise.c
+ CC src/isa/riscv64/instr/rvt/convert.c
+ CC src/isa/riscv64/instr/rvt/conv2d.c
+ CC src/isa/riscv64/instr/rvt/type.c
+ CC src/isa/riscv64/instr/rvt/tensor_init.c
+ CC src/isa/riscv64/instr/rvt/tensor_copy.c
+ CC src/isa/riscv64/instr/rvt/pooling.c
+ CC src/isa/riscv64/instr/rvt/register.c
+ CC src/isa/riscv64/instr/rvt/matmul.c
+ CC src/isa/riscv64/instr/vp.c
+ CC src/engine/interpreter/hostcall.c
+ CC src/engine/interpreter/fp.c
+ CC src/engine/interpreter/init.c
+ CC src/device/io/mmio.c
+ CC src/device/io/port-io.c
+ CC src/device/io/map.c
+ CC src/profiling/profiling_control.c
+ CC src/checkpoint/cpt_env.c
+ CXX src/checkpoint/simpoint.cpp
+ CXX src/checkpoint/path_manager.cpp
+ CXX src/checkpoint/serializer.cpp
+ CXX src/checkpoint/semantic_point.cpp
+ CXX src/base/output.cpp
+ CXX src/iostream3/zfstream.cpp
+ CXX src/cpu/difftest/ame/msync_queue_wrapper.cpp
+ CXX src/cpu/difftest/ame/amu_ctrl_queue_wrapper.cpp
+ CXX src/memory/store_queue_wrapper.cpp
+ CXX src/memory/elfloader.cpp
SPECIALIZE_TYPE=RISCV SOFTFLOAT_OPTS="-DSOFTFLOAT_ROUND_ODD -DINLINE_LEVEL=5
-DSOFTFLOAT_FAST_DIV32TO16 -DSOFTFLOAT_FAST_DIV64TO32 -fPIC" make -s -C
/home/riscv/riscv-lab/xs-env/NEMU/resource/softfloat/repo/build/Linux-x86_64-GCC all
mkdir -p resource/softfloat/build
ln -sf /home/riscv/riscv-lab/xs-env/NEMU/resource/softfloat/repo/build/Linux-x86_64-GCC/softfloat.a
resource/softfloat/build/softfloat.a
+ g++ /home/riscv/riscv-lab/xs-env/NEMU/build/riscv64-nemu-interpreter-so
        Command being timed: "make -j1"
        User time (seconds): 22.97
        System time (seconds): 7.18
        Percent of CPU this job got: 101%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:29.76
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 445324
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 1324
        Minor (reclaiming a frame) page faults: 1488436
        Voluntary context switches: 7740
        Involuntary context switches: 491
        Swaps: 0
        File system inputs: 227800
        File system outputs: 96112
        Socket messages sent: 0
```

```
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:14: cp .config /home/riscv/riscv-lab/stage1-evidence/nemu.config
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:15: file build/riscv64-nemu-interpreter-so
build/riscv64-nemu-interpreter-so: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically
linked, BuildID[sha1]=4841e67452e47681eecaff72b0e3526c02cec25f, not stripped
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:16: sha256sum build/riscv64-nemu-interpreter-so
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:17: date -Is
2026-10-09T20:25:12+08:00
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:18: echo NEMU_BUILD_PASS
NEMU_BUILD_PASS
```

1.12 EMU 构建原始日志

以下为对应历史记录的原始输出，日期、路径和提交身份保留原值。

```
+ /home/riscv/riscv-lab/stage1-build-emu.sh:12: date -Is
2026-10-09T20:42:28+08:00
+ /home/riscv/riscv-lab/stage1-build-emu.sh:13: cd /home/riscv/riscv-lab/xs-env/XiangShan
+ /home/riscv/riscv-lab/stage1-build-emu.sh:14: git rev-parse HEAD
16ae9ddcda54fc9a2fddffad73174cf793ac7814
+ /home/riscv/riscv-lab/stage1-build-emu.sh:15: free -h
               total        used        free      shared  buff/cache   available
Mem:            19Gi       623Mi        16Gi       2.0Mi       2.3Gi        18Gi
Swap:           16Gi        73Mi        15Gi
+ /home/riscv/riscv-lab/stage1-build-emu.sh:16: /usr/bin/time -v make emu CONFIG=MinimalConfig
NUM_CORES=1 EMU_THREADS=1 EMU_TRACE=fst JVM_XMX=12G -j4
make -C ./difftest emu SIM_TOP=SimTop DESIGN_DIR=/home/riscv/riscv-lab/xs-env/XiangShan
NUM_CORES=1 RTL_SUFFIX=sv
make[1]: Entering directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
Fri, 09 Oct 2026 20:42:28 +0800
Building emu...
make[2]: Entering directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
time -avp -o /home/riscv/riscv-lab/xs-env/XiangShan/build/time.log make -s VM_PARALLEL_BUILDS=1
OPT_SLOW="-O0" \
                        OPT_FAST=-O3 \
                        PGO_CFLAGS= \
                        PGO_LDFLAGS= \
                        -C /home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile -f
                        /home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile/VSimTop.mk
make[3]: Entering directory '/home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile'
make[3]: Leaving directory '/home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile'
make[2]: Leaving directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
make[1]: Leaving directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
        Command being timed: "make emu CONFIG=MinimalConfig NUM_CORES=1 EMU_THREADS=1
        EMU_TRACE=fst JVM_XMX=12G -j4"
        User time (seconds): 2451.21
        System time (seconds): 187.35
        Percent of CPU this job got: 397%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 11:03.88
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
```

```
        Maximum resident set size (kbytes): 1301648
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 479
        Minor (reclaiming a frame) page faults: 50883606
        Voluntary context switches: 5000
        Involuntary context switches: 4568
        Swaps: 0
        File system inputs: 8832
        File system outputs: 3129248
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
+ /home/riscv/riscv-lab/stage1-build-emu.sh:17: ./build/emu --help
+ /home/riscv/riscv-lab/stage1-build-emu.sh:18: sha256sum build/emu
+ /home/riscv/riscv-lab/stage1-build-emu.sh:19: cp build/time.log
/home/riscv/riscv-lab/stage1-evidence/emu-time.log
+ /home/riscv/riscv-lab/stage1-build-emu.sh:20: date -Is
2026-10-09T20:53:32+08:00
+ /home/riscv/riscv-lab/stage1-build-emu.sh:21: echo XIANGSHAN_EMU_BUILD_PASS
XIANGSHAN_EMU_BUILD_PASS
+ /home/riscv/riscv-lab/stage1-build-emu.sh:1: rc=0
++ /home/riscv/riscv-lab/stage1-build-emu.sh:1: date -Is
+ /home/riscv/riscv-lab/stage1-build-emu.sh:1: printf 'exit_code=%s end=%s\n' 0 2026-10-09T20:53:32+08:00
```

1.13 工具与编译链版本原始输出

以下为对应历史记录的原始输出，日期、路径和提交身份保留原值。

```
java 21.0.12 2026-07-21 LTS
Java(TM) SE Runtime Environment Oracle GraalVM 21.0.12+7.1 (build 21.0.12+7-LTS-jvmci-23.1-b96)
Java HotSpot(TM) 64-Bit Server VM Oracle GraalVM 21.0.12+7.1 (build 21.0.12+7-LTS-jvmci-23.1-b96,
mixed mode, sharing)
javac 21.0.12
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

g++ (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Ubuntu clang version 19.1.7 (++20250114103320+cd708029e0b2-1~exp1~20250114103432.75)
Target: x86_64-pc-linux-gnu
Thread model: posix
InstalledDir: /usr/lib/llvm-19/bin
Verilator 5.052 2026-09-05 rev v5.052
LLVM (http://llvm.org/):
  LLVM version 18.0.0git
  Optimized build.
CIRCT firtool-1.62.1
riscv64-linux-gnu-gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
```

```
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

GNU assembler (GNU Binutils for Ubuntu) 2.38
Copyright (C) 2022 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or later.
This program has absolutely no warranty.
This assembler was configured for a target of `riscv64-linux-gnu'.
Mill Build Tool version 0.12.3
Java version: 21.0.12, vendor: Oracle Corporation, runtime: /opt/graalvm-jdk-21
Default locale: en, platform encoding: UTF-8
OS name: "Linux", version: 6.6.87.2-microsoft-standard-WSL2, arch: amd64

[Scala/Chisel dependency versions]
val defaultScalaVersion = "2.13.15"
 "chisel"        -> ivy"org.chipsalliance::chisel:6.6.0",
 "chisel-plugin" -> ivy"org.chipsalliance:::chisel-plugin:6.6.0",
 "chiseltest"    -> ivy"edu.berkeley.cs::chiseltest:6.0.0"
 override def scalaVersion = defaultScalaVersion
 def scalaVersion: T[String] = T(defaultScalaVersion)
   def scalaVersion: T[String] = T(defaultScalaVersion)
   def scalaReflectIvy = ivy"org.scala-lang:scala-reflect:${defaultScalaVersion}"
   def scalaVersion: T[String] = T(defaultScalaVersion)
   def scalaVersion: T[String] = T(defaultScalaVersion)
 override def scalaVersion: T[String] = T(defaultScalaVersion)
 override def ivyDeps = super.ivyDeps() ++ Agg(ivy"org.scala-lang:scala-reflect:${defaultScalaVersion}")
 def scalaReflectIvy = ivy"org.scala-lang:scala-reflect:${defaultScalaVersion}"
```

1.14 固定子模块版本清单

以下为对应历史记录的原始输出，日期、路径和提交身份保留原值。

```
1d2a9bf6da8e050d975ef2669c42faa93943f489 DRAMsim3 (heads/cosim-kmh)
68739c5a23cc48c9a13d28793834e4fee95d33ee DRAMsim3/ext/SuperLU_MT_3.1 (68739c5)
a48a11f208ff81ad765591d83271d1b339fb3861 NEMU (a48a11f)
e51a936d7fddfa03c81692039f184ab6c437e99e NEMU/resource/simpoint/simpoint_repo (heads/main)
bba8f3a4e3a15ef135baed37566be3e8a0c7d384 NutShell (bba8f3a)
29e80f8525735d1cb77f30fd2a3ea5eff3e6528c NutShell/difftest (29e80f8)
16ae9ddcda54fc9a2fddffad73174cf793ac7814 XiangShan (16ae9dd)
82079f4081bbc1f0221c790fbf8a29b1f23f48b8 XiangShan/coupledL2 (82079f4)
8faa17c9f7115782da1bd1665bd95587fba525ec XiangShan/coupledL2/HuanCun (8faa17c)
de50694f6ffb48780a4b4665907442b3e4dd0d6d XiangShan/coupledL2/HuanCun/Utility (de50694)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/coupledL2/HuanCun/Utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/HuanCun/Utility/rocket-chip/cde
(52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/HuanCun/Utility/rocket-chip/hardfloat
(d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/HuanCun/Utility/rocket-chip/torture
(99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/HuanCun/Utility/rocket-chip/torture/env
(ce70afb)
```

90ae9a9314d444ade097fd5a471916869559b4fd XiangShan/coupledL2/HuanCun/rocket-chip (90ae9a9)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/HuanCun/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/HuanCun/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/HuanCun/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/HuanCun/rocket-chip/torture/env (ce70afb)
90ae9a9314d444ade097fd5a471916869559b4fd XiangShan/coupledL2/rocket-chip (90ae9a9)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/coupledL2/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/coupledL2/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/rocket-chip/torture/env (ce70afb)
992ce71166405de47c93102100a897726bd93bac XiangShan/coupledL2/utility (992ce71)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/coupledL2/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/coupledL2/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/coupledL2/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/utility/rocket-chip/torture/env (ce70afb)
62e4540c03c03aa947bdb67f4b6bf66f1a29c5d0 XiangShan/difftest (62e4540)
e1bd4695ca7beb36a5ce7357e9527ad9e95b9ec1 XiangShan/fudian (heads/pipeline)
b64af41c3276f97f0e181920400ee056b9c88037 XiangShan/fudian/berkeley-softfloat-3 (b64af41)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/fudian/berkeley-testfloat-3 (06b2007)
90aaf5935206ff322e461c3d021436c20dd0ac85 XiangShan/huancun (90aaf59)
de50694f6ffb48780a4b4665907442b3e4dd0d6d XiangShan/huancun/Utility (de50694)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/huancun/Utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/huancun/Utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/huancun/Utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/huancun/Utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/huancun/Utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/huancun/Utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/huancun/Utility/rocket-chip/torture/env (ce70afb)
e64b744b1490992bd038aaf9ca628f0e29fa0e59 XiangShan/huancun/rocket-chip (e64b744)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/huancun/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/huancun/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/huancun/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/huancun/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/huancun/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/huancun/rocket-chip/torture/env (ce70afb)
614ceb4c5d39d32e6ae786693a35da429ea32c12 XiangShan/openLLC (614ceb4)
d3b04fef92092f9b27023bea57cc2ca99a8ffea0 XiangShan/openLLC/coupledL2 (d3b04fe)
90aaf5935206ff322e461c3d021436c20dd0ac85 XiangShan/openLLC/coupledL2/HuanCun (90aaf59)
de50694f6ffb48780a4b4665907442b3e4dd0d6d XiangShan/openLLC/coupledL2/HuanCun/Utility (de50694)

175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/torture/env (ce70afb)
e64b744b1490992bd038aaf9ca628f0e29fa0e59 XiangShan/openLLC/coupledL2/HuanCun/rocket-chip (e64b744)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/torture/env (ce70afb)
bb4baf85c5bd4b55ffdcda12a75648fef212ab69 XiangShan/openLLC/coupledL2/rocket-chip (bb4baf8)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/coupledL2/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/coupledL2/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/openLLC/coupledL2/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/openLLC/coupledL2/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/coupledL2/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/coupledL2/rocket-chip/torture/env (ce70afb)
342e0ad98bf24bf550f4d44099bd64875145d07c XiangShan/openLLC/coupledL2/utility (342e0ad)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/openLLC/coupledL2/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/coupledL2/utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/coupledL2/utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/openLLC/coupledL2/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/openLLC/coupledL2/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/coupledL2/utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/coupledL2/utility/rocket-chip/torture/env (ce70afb)
546a621ed82302db1bd1edc900b17ba55682c15c XiangShan/openLLC/openNCB (546a621)
c0955429aa3d9fe2be560f28599ece700fb0466c XiangShan/openLLC/openNCB/external/rocket-chip (c095542)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/openNCB/external/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/openNCB/external/rocket-chip/hardfloat (d93aa57)

5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/openNCB/external/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/openNCB/external/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/openNCB/external/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/openNCB/external/rocket-chip/torture/env (ce70afb)
bb4baf85c5bd4b55ffdcda12a75648fef212ab69 XiangShan/openLLC/rocket-chip (bb4baf8)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/openLLC/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/rocket-chip/torture/env (ce70afb)
342e0ad98bf24bf550f4d44099bd64875145d07c XiangShan/openLLC/utility (342e0ad)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/openLLC/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/utility/rocket-chip/torture/env (ce70afb)
ed8c30e1ec8a9c9aa6b116ce54281235d00273fa XiangShan/ready-to-run (ed8c30e)
bea0af5d855a871699b907c1988961141bbf28e7 XiangShan/rocket-chip (bea0af5)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/rocket-chip/torture/env (ce70afb)
9464aee3c77b021a037f28aa1b3fe53c71b516f6 XiangShan/src/main/resources/aia (heads/main)
a8ca0c6564865ee69b564d431ecbc52074ba8060 XiangShan/utility (a8ca0c6)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/utility/rocket-chip/torture/env (ce70afb)
cadd3c2f43096e253d61296b33ac697be8354e29 XiangShan/yunsuan (cadd3c2)
086a238062fdfb3281b745a537b361983d0154b5 nexus-am (086a238)

1.15 原始波形事件记录

以下为对应历史记录的原始输出，日期、路径和提交身份保留原值。

cycle,scope,pc,instruction,vecwen,wdest
19562,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000012c,0x」
cc0472d7,0,5

19624,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000130,0x
2050087,1,1
19624,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000134,
0x2058107,1,2
19631,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000138,0x
21101d7,1,3
19646,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000013c,0x
20601a7,1,32
19646,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000140,
0x28513,0,10
19647,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000142,0x
8067,0,0
25442,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000012c,0x
cc0472d7,0,5
25458,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000130,0x
2050087,1,1
25459,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000134,0x
2058107,1,2
25466,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000138,0x
21101d7,1,3
25481,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000013c,0x
20601a7,1,32
25481,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000140,
0x28513,0,10
25481,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_2.dpic,0x80000142,
0x8067,0,0
32991,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000012c,0x
cc0472d7,0,5
33007,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000130,0x
2050087,1,1
33008,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000134,0x
2058107,1,2
33015,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000138,0x
21101d7,1,3
33030,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000013c,0x
20601a7,1,32
33030,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000140,
0x28513,0,10
33030,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_2.dpic,0x80000142,
0x8067,0,0

1.16 向量运算单元事件记录

以下为对应历史记录的原始输出，日期、路径和提交身份保留原值。

cycle,scope,phase,fuOpType,vsew,src0,src1,result
19625,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,in,64,0,0xffffffffffff
fff0102030405060708,0xffffffffffffffff0807060504030201,
19626,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,out,,,,,0xffffffffffffff
f0909090909090909
25460,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,in,64,0,0xffffffffffff
ffff8f9fafbfcfdfeff,0xffffffffffffffff8f9fafbfcfdfeff,
25461,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,out,,,,,0xffffffffffffff
ff0f2f4f6f8fafcfe
33009,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,in,64,0,0xffffffffffff
fffff7f80ff0001ff01,0xffffffffffffffff7f800100ff807f,

33010,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,out,,,,,0xffffffffffffffff」
ffefe000000007f80